

SISTEMAS DISTRIBUIDOS · GRUPO 2 · SUSTENTACIÓN FINAL

BarberSaaS

Una plataforma de microservicios para administrar muchas barberías desde una sola app

release v3.0.0 · noviembre 2026

Carolay Arraut · Daniel Cerquera · Juan Pablo Borrero · Carlos Leal

Corporación Universitaria del Huila — CORHUILA

Las barberías se administran a mano

Hoy

Citas por llamada o WhatsApp

Filas, olvidos y clientes que se van

Ingresos y gastos en papel

Sin saber cuánto produce cada barbero

Ningún programa de fidelidad

El cliente vuelve solo por costumbre

Nuestra respuesta

El cliente reserva sin llamar

Ve las franjas libres y elige; nunca dos en la misma

El dueño ve su negocio

Finanzas, inventario, reseñas y agenda en la app

Fidelidad automática

Un sello por cita completada y cupón al completar la tarjeta

Cuatro roles, una sola app



Cliente

Busca barberías
Reserva y cancela
Califica la cita
Junta sellos y cupones



Barbero

Ve su agenda
Confirma, inicia y completa citas
Consulta sus horarios



Dueño

Servicios, barberos y horarios
Finanzas e inventario
Fidelidad y reseñas
Paga su plan



Super admin

Ve todas las barberías
Las suspende o reactiva
Administra los planes
Resumen de la plataforma



Cada rol ve pestañas distintas, pero ocultar una pestaña no es seguridad: cada servicio revisa el rol y la barbería del token.

Un sistema distribuido de verdad

30

repositorios

8 dominios × (db, api, app) + 6 transversales

8

dominios de negocio

cada uno dueño de sus datos

2 + 2

lenguajes y frameworks

Java + Python · React + Angular

20

decisiones de arquitectura

escritas como ADR en DOCS

31/31

pruebas de punta a punta

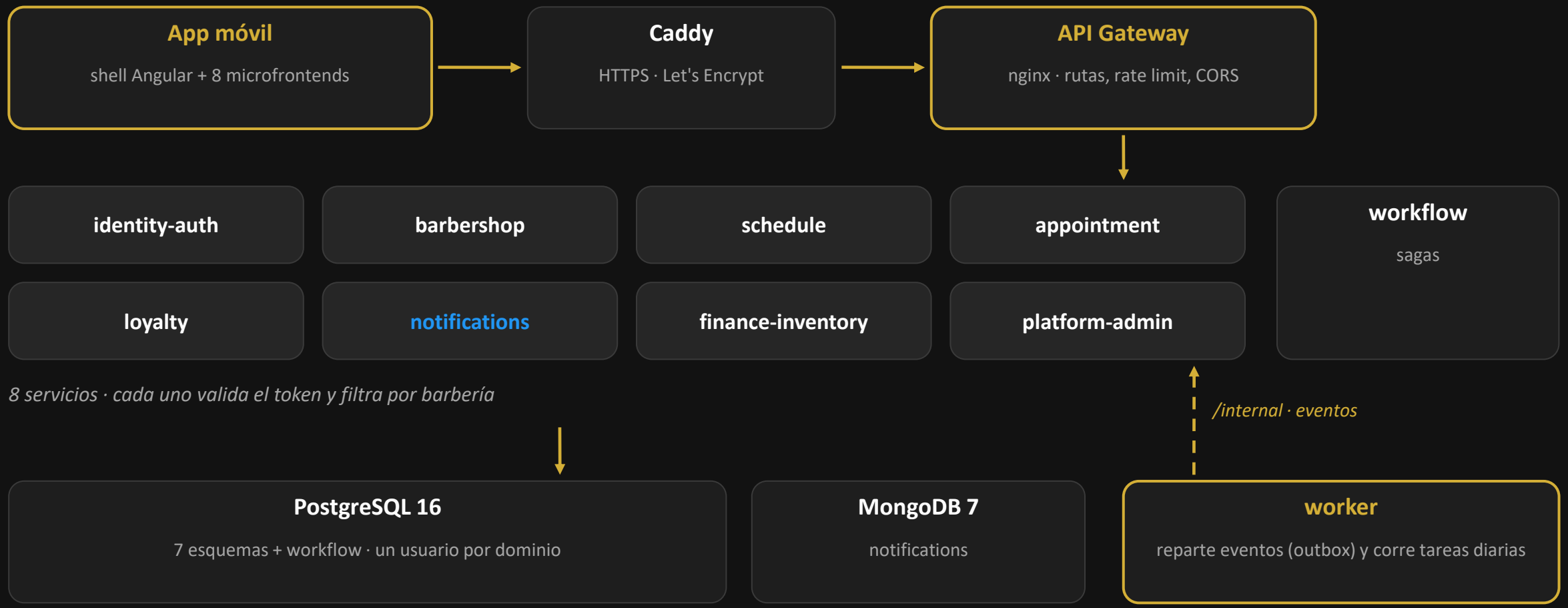
contra producción en Azure

14

contenedores en producción

corriendo en Azure con HTTPS

Microservicios detrás de un solo punto de entrada



Externos: Firebase (push) · SMTP (correo) · Wompi (pagos)

Ocho servicios, cada uno con un dueño

identity-auth

Registro, correo verificado, login, tokens

Daniel — Java · React

barbershop

Barbería, servicios, barberos, catálogo

Carlos — Java · React

schedule

Horarios, excepciones, disponibilidad

Carlos — Java · React

appointment

Citas, anti doble reserva, reseñas

Juan Pablo — Java · React

loyalty

Sellos, canje y cupones

Juan Pablo — Java · Angular

notifications

Bandeja, push y correos

Carlos — Python · Angular

finance-inventory

Ingresos, gastos e inventario

Juan Pablo — Java · Angular

platform-admin

Planes, pagos, suspensión

Daniel — Java · Angular

El stack y por qué lo elegimos

Backend

- Java 21 · Spring Boot 3.5
- Python 3.12 · FastAPI
- Arquitectura hexagonal

ADR-005 · ADR-012

Datos

- PostgreSQL 16 · 7 esquemas
- MongoDB 7 · replica set
- Liquibase en cada -db

ADR-006 · 007 · 011

App

- Ionic 8 + Capacitor 7
- Shell Angular 21
- 4 remotos React + 4 Angular

ADR-013

Plataforma

- Docker Compose · nginx
- Azure VM + Caddy HTTPS
- Firebase · SMTP · Wompi

ADR-016 · ADR-020

Por qué así: el Anexo J del curso pide una base por motor, dos lenguajes de backend y microfrontends React y Angular. Sin Kafka: el outbox con el worker no pierde eventos y es más simple de operar.

20 ADRs: cada decisión con su porqué

ADR-004 — **Microservicios completos**

8 dominios, cada uno con su API, su app y su esquema (reemplaza al monolito de ADR-002)

ADR-011 — **Una instancia por motor**

PostgreSQL con un esquema por dominio y MongoDB para notifications; sin base compartida entre dominios

ADR-012 — **Dos lenguajes de backend**

Java 21 en 8 servicios; Python en notifications-api y worker

ADR-009 — **Saga en su propio esquema**

workflow guarda el estado de la saga de registro de barbería

ADR-016 — **Outbox + worker**

Eventos por HTTP interno con reintentos; sin broker que operar

ADR-013 — **App híbrida**

Ionic + Capacitor, shell Angular, microfrontends React y Angular

ADR-017 · 018 — **Pagos y verificación**

Wompi sandbox con evento firmado; código de 6 dígitos al correo

ADR-020 — **Producción en Azure**

Una VM con Compose y HTTPS; reemplaza a Oracle (ADR-019)

Formato Contexto → Decisión → Consecuencias · 5 reemplazados por uno nuevo · 05-architecture/decisions

Una app híbrida hecha de microfrontends



Shell Angular (front)

Sesión, cliente HTTP con el token, pestañas por rol, push, APK

- identity-auth · React
- barbershop · React
- schedule · React
- appointment · React
- loyalty · Angular
- notifications · Angular
- finance-inv. · Angular
- platform-admin · Angular

Pestañas según el rol

Cliente

Buscar · Mis citas · Fidelidad · Avisos · Perfil

Barbero

Mi agenda · Horarios · Avisos · Perfil

Dueño

Agenda · Mi barbería · Horarios · Finanzas · Avisos · Perfil

Super admin

Plataforma · Avisos · Perfil

Reservar una cita, de punta a punta



1 · Busca

Catálogo público de barberías, sin token

2 · Entra

Token de barbería con su barbershopId

3 · Elige franja

schedule: horario – excepciones – ocupadas

4 · Reserva

appointment: precio fijo, bloqueo de fila

5 · Le avisan

Evento → worker → push y correo

Dos clientes, la misma franja

El primero la obtiene; el segundo recibe 409. Lo comprueba la prueba e2e en producción.

Después de la cita

Al completarla: un sello de fidelidad, el ingreso en finanzas y un aviso. El cliente la califica una vez.

Registrar una barbería toca tres servicios

1 `create-barbershop`
barbershop-api
La barbería nace en prueba de 60 días

→

2 `assign-plan`
platform-admin-api
Se le asigna el plan elegido

→

3 `create-owner`
identity-auth-api
Se crea la cuenta del dueño

← *Si un paso falla, se compensan los anteriores: nunca queda una barbería sin dueño*

Estado guardado
Después de cada paso, en el esquema workflow (ADR-009)

Reintento seguro
Idempotency-Key: <sagaId>:<paso> en cada llamada

Correo verificado
Al final, el dueño recibe su código de 6 dígitos

Outbox + worker: eventos sin perder ninguno

1 · El productor

Cambia sus datos y guarda el evento en su tabla outbox en la misma transacción

2 · El worker

Cada 5 s lee hasta 50 eventos con un token por servicio y los entrega por HTTP interno

3 · Los consumidores

Son idempotentes: si un evento llega dos veces, la segunda no hace nada

Evento	Productor	Consumidores
AppointmentCompleted	appointment	loyalty · notifications · finance
AppointmentConfirmed / Cancelled / ReminderDue	appointment	notifications
StickerGranted · RewardRedeemed	loyalty	notifications
EmailVerificationRequested · PasswordResetRequested	identity-auth	notifications

Seguridad por capas, revisada con STRIDE

Borde	HTTPS con Let's Encrypt · solo puertos 80 y 443 · SSH solo para una IP
Gateway	Límite de tasa por IP y global en login · CORS solo para la app · correlación
Token JWT RS256	Firmado con llave privada · 15 minutos · refresh que rota · logout real
Audiencia y scopes	Tokens de servicio por destino, con permisos mínimos y 7 días de vida
Multi-tenant	El barbershopId sale del token: otra barbería recibe 404
Threat models	4 fronteras analizadas con STRIDE · ningún P0 ni P1 abierto

Lo que revisa cada servicio

1. Algoritmo RS256 y firma válida
2. No vencido · emisor correcto
3. aud = barber-saas-app en /api
4. El rol de la operación
5. Scope y servicio en /internal
6. Filtro por barbershopId

alg: none, HS256 u otra llave → 401

Del portátil a producción con los mismos archivos

Local

portátil

- Docker Desktop en el portátil
- clone-all.sh · dev-keys.sh
- up.sh dev: bases, migraciones y 14 contenedores

QA

validación

- Lo validado para salir
- Probado antes de salir
- Es lo que corre hoy en Azure

Producción

nube Azure

- VM Azure for Students (2 vCPU, 8 GB)
- Caddy + HTTPS · backup diario
- barbersaas.chilecentral.cloudapp.azure.com

98 USD de 100 de crédito restantes

~45 s arranque al encender la VM

40 s rollback ensayado

Se cae un servicio y el resto sigue

1. Flujo normal

Catálogo, reserva y fidelidad responden por el gateway

2. Inyectamos el fallo

docker stop loyalty-api · docker stop notifications-api

3. Degradación controlada

Solo esa ruta responde 503 SERVICE_UNAVAILABLE con su traceId; el registro y el catálogo siguen

4. Recuperación

docker start: la ruta vuelve sola y el worker entrega el evento que quedó en el outbox

Medido sobre v3.0.0 · 12-nov-2026

2,0 s	respuesta 503 de la ruta caída
13 ms	otras rutas mientras tanto
4 s	la ruta vuelve tras docker start
201	registro con notifications caído
7 s	evento entregado tras la recuperación

Registro completo de la corrida: documentation/failure-path.md

Cómo resiste y cómo se observa

Resiliencia

- Gateway: DNS por petición, una ruta caída no tumba las demás
- Timeouts: 2 s para conectar, 10 s para responder
- Límite por cliente: 10 req/s (ráfaga 20) → 429
- Worker: 8 reintentos con backoff 5 s → 300 s y jitter
- Idempotencia por id de evento; dead letter tras 8 intentos o un 4xx

Observabilidad

- /health en cada servicio y en el gateway
- X-Correlation-Id de punta a punta y traceId en cada error
- Logs por contenedor con docker compose logs
- Perfil observability: OpenTelemetry, Prometheus, Grafana
- Aún sin métricas exportadas por los servicios

Números

- e2e en Azure: 31/31 pasos
- Tests: notifications 191 · identity-auth 111 · platform-admin 81 · worker 33
- Arranque de la VM: ~45 s · rollback ensayado: 40 s
- Fallo inyectado: 2,0 s de 503 y 7 s para entregar el evento

Probado en producción, auditado con M1–M9

31/31

pasos de la prueba de punta a punta en Azure

Saga · verificación · reserva · doble reserva · cita completa · reseña · aislamiento · suspensión · eventos entregados

Unitarias

Reglas y casos de uso, sin red

HTTP

Códigos, errores, tokens falsos, otro dueño → 404

Por servicio

notifications 191 · identity-auth 111 · platform-admin 81 · worker 33

Lista M1–M9

M1 cumple	**M2** cumple	**M3** cumple
M4 cumple	**M5** cumple	**M6** cumple
M7 cumple	**M8** cumple	**M9** cumple

M2: gitleaks sobre el historial de 32 repos, 0 secretos reales · M4: arranque desde cero con los tags v3.0.0, 13 contenedores sanos en 167 s

La prueba real encontró lo que los tests no

notifications respondía 401 a todo

PyJWT rechaza un token con audiencia si no se le dice cuál esperar. No habrían llegado push, bandeja ni códigos de verificación.

Corregido el 11-nov

Audiencia revisada por ruta · 191 tests · e2e 31/31

platform-admin aceptaba cualquier token del worker

El outbox y las tareas diarias no exigían su permiso. Un token filtrado de otro servicio servía allí.

Corregido el 11-nov

Scopes outbox:relay y jobs:run · tope de 50 · 81 tests

Lo que falta, dicho con su razón

Pago real con Wompi	Exige una cuenta de comercio activa. En producción nada se paga sin el evento firmado; la demo usa el simulador
Carga, chaos automatizado y Testcontainers	El fallo se inyectó a mano (ver «Demo en vivo · con un fallo»); las pruebas de carga y de chaos automatizadas no se corrieron
Reintento de eventos fallidos	Un evento rechazado queda como dead letter; no se pierde, pero no se reintenta solo
Métricas de los servicios	La pila OpenTelemetry + Prometheus + Grafana existe, pero los servicios aún no exportan métricas



Gracias

¿Preguntas?

Distribuido de verdad

8 dominios, 30 repos, 2 lenguajes

Seguro por capas

JWT RS256, scopes, multi-tenant, STRIDE

En producción

Azure con HTTPS y e2e 31/31

Carolay Arraut · Daniel Cerquera · Juan Pablo Borrero · Carlos Leal